TP 2 - Outils de manipulation et traitement d'images en Python (partie 2)

Parcours d'une image:

Le code suivant nous montre comment accéder aux pixels d'une image avec OpenCV :

```
import cv2

# Read the image
img = cv2.imread('cablecar.bmp')

if image is None:
   print("Erreur : Impossible de charger l'image.")
   else:
      # Get the image height and width
      height, width, _ = img.shape
for y in range(height):
   for x in range(width):
      pixel = image[y, x]
      B, G, R = pixel
      if 100 < x < 200 and 100 < y < 150 :
         img[y, x] = (255,0,0)
# Show the image
```

On peut également accéder à une région de l'image et la modifier avec Numpy:

```
# ...
img[100:150, 100:200] = (255,0,0)
```

Parcours d'une image bloc par bloc

Le code suivant permet de diviser l'image en bloc et de les afficher

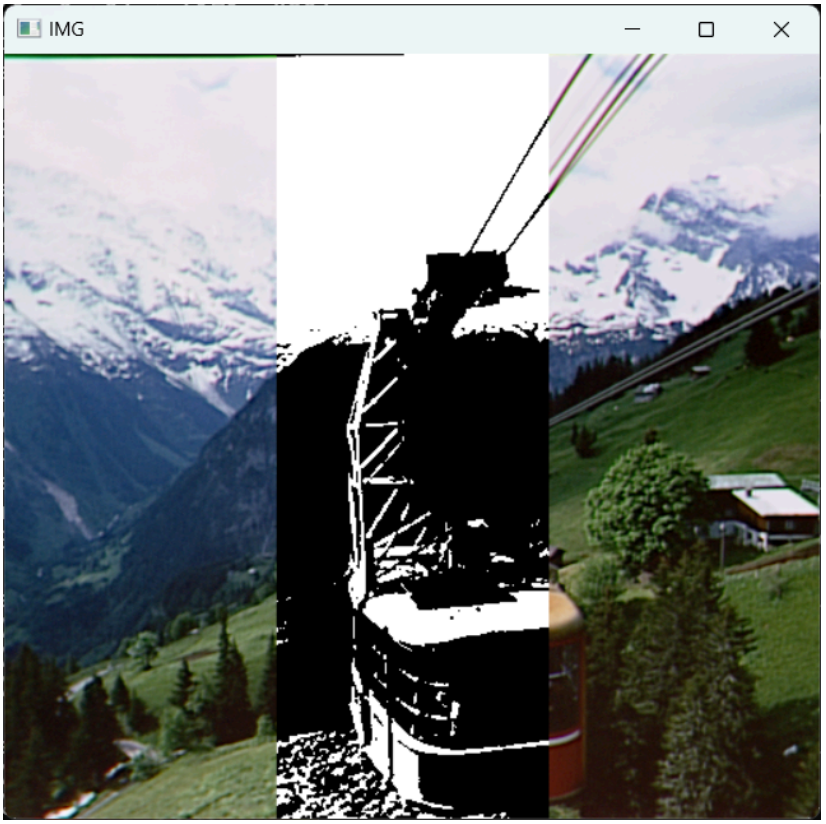
```
# Définir la taille des blocs
block_size = int(width/2)
# Parcourir l'image bloc par bloc
for y in range(0, height, block_size):
   for x in range(0, width, block_size):
      # Définir les limites du bloc
      y_end = min(y + block_size, height)
      x_end = min(x + block_size, width)
      # Extraire le bloc
      block = img[y:y_end, x:x_end]
      # Afficher les coordonnées du bloc
      print(f"Bloc ({x}, {y}) → ({x_end}, {y_end})")
      # Afficher chaque bloc
      cv2.imshow("Bloc", block)
      cv2.waitKey(100)

cv2.destroyAllWindows()
```

La binarisation d'une image:

Dans OpenCV, la fonction cv2.threshold nous permet d'effectuer une binarisation d'une image en niveau de gris. Le seuil de cette dernière peut être déterminé soit manuellement ou automatiquement. Une des méthodes de seuillage automatique les plus utilisées est la méthode d'Otsu. Celle-ci est basée sur le calcul de l'histogramme de l'image, ainsi que la minimisation de la variance intra-classe (sachant qu'il y a deux classes : la classe objets et la classe background).

```
# %%
threshold = 127
_, img_bw = cv2.threshold(gray_img, threshold, 255, cv2.THRESH_BINARY)
threshold, img_bw_otsu = cv2.threshold(gray_img, threshold, 255, cv2.THRESH_OTSU)
print (f"Otsu threshold {threshold}")
figure, plots = plt.subplots(1, 2)
plots[0].imshow(img_bw, cmap='gray')
plots[0].axis('off')
plots[0].set_title('Binarized Image')
plots[1].imshow(img_bw_otsu, cmap='gray')
plots[1].axis('off')
plots[1].set_title('Otsu Binarized Image')
plt.show()
```

Exercice:

Lire une image, la diviser en trois blocs verticaux et binariser le bloc du milieu comme indiqué sur l'image ci dessous: